

The first option seems to be good for those who don't want to engage in exploring the GSL source, but want to get their work done as easily as possible. The most obvious requirement is the knowledge to create a Makefile, since handling so many files with manual compilation is not a user-friendly option. Personally, I would prefer this option, because it is less troublesome.

The second option could be more suitable if you want all the functions in one file, and you then wish to just 'forget about it'. I'll show you the result of my applying this approach to one of my programs.

I advise you to write your programs normally first, and then make them portable. The obvious reason is because you will then already know which header files (and by extension, .c files) contain your desired type definitions and function definitions. I suggest the following steps:

- First write programs normally, and check that you obtain the expected results.
- Add GSL function definitions and type definitions to your own source code; then remove the header file declarations. Verify that it still works as expected by compiling (and running) the program.
- Most of the work is done in the second step itself, but some additional items like global variables, macros, etc, should also be included in the program file.
- Finally, you can decide whether the entire code should be in one file, or if you want to split it into separate files.

For an example, look at the program below. My previous version included two header files: `"#include <gsl/gsl_complex.h>"` and `"#include <gsl/gsl_complex_math.h>"`. I have made this version portable by copying the GSL definitions into it:

```
#include <stdio.h>
#include <math.h>

#define GSL_SET_COMPLEX(zp, x, y) do {(zp)->dat[0]=(x); (zp)->dat[1]=(y);} while(0)

#define GSL_REAL(z)      ((z).dat[0])
#define GSL_IMAG(z)      ((z).dat[1])

typedef struct
{
    double dat[2];
}
gsl_complex;

gsl_complex gsl_complex_polar (double r, double theta)
{
    gsl_complex z;
    GSL_SET_COMPLEX (&z, r * cos (theta), r * sin (theta));
    return z;
}

double gsl_complex_abs (gsl_complex z)
{

```

```
    return hypot (GSL_REAL (z), GSL_IMAG (z));
}

int main (void)
{
    double x = 5.0;
    double y = 0.785;
    gsl_complex z;
    z = gsl_complex_polar (x, y);
    printf("The complex number is:\n");
    printf("%0.18f + i %0.18f\n", z.dat[0], z.dat[1]);

    double abs;
    abs=gsl_complex_abs(z);
    printf("The magnitude of the complex number is:\n");
    printf("%0.18f\n",abs);
    return 0;
}
```

Compiling and executing the program:

```
$ gcc -Wall demo_gsl.c -lgsl -lgslcblas -lm -o demo_gsl
$ ./demo_gsl
The complex number is:
3.535173840839293408 + i 3.535893934360332381
The magnitude of the complex number is:
5.000000000000000000
```

I would suggest you always make your programs portable if you are going to distribute them to somebody else. Also, it might enhance your code's re-usability.

Errors in GSL

Most of the syntax errors and run-time errors are handled by GCC. GSL is just an API written in C. Errors particular to the function/routine are displayed accordingly. The most impressive thing in GSL is that you can write your own error functions (I've included a brief demonstration below).

Before that, I want to mention one run-time error that I encountered, which you'll face if you did not follow the standard installation procedure and installed GSL at a different location:

```
error while loading shared libraries: libgsl.so.8: cannot open shared
object file: No such file or directory.
```

The solution is to either specify, in your GCC invocations, the path where you installed GSL; or use the environment variable `LD_LIBRARY_PATH` to indicate the path to the library.

You can specify the path (for the given usages) as follows (replace the path with the path specific to your installation):

Compiling: `gcc -Wall -I/usr/local/include -c first_gsl.c`

Linking: `gcc -L/usr/local/lib first_gsl.o -lgsl -lgslcblas -lm`

Running: `./a.out`

To use the environment variable instead, execute the following commands (for the Bash shell):